

Building LLM - General Project Report

Introduction

Building LLM is an educational project that begins with the smallest understandable language model and evolves one version at a time toward a modern small LLM.

Each version introduces a focused concept while keeping the mechanisms visible, executable, and suitable for study.

one version -> one concept -> one verifiable result

Project goal

The goal is not to compete with industrial models. The goal is to understand how language models are built, trained, evaluated, and used, progressing from direct statistical counts to a decoder-only Transformer.

The project follows these principles:

- English is the initial modeling language;
 - early mechanisms are implemented from first principles;
 - complexity increases gradually and remains explainable;
 - notebooks run from top to bottom;
 - reproducibility matters more than model scale;
 - advanced training targets accessible Kaggle hardware.
-

Project constraints

- Version 1 uses standard Python only;
- NumPy is introduced before a machine-learning framework;
- TensorFlow appears only after a small neural model is understood;
- public datasets require documented origin, version, and license;
- every version keeps a quick check that does not require a GPU.

What LLM means in this journey

The first versions are language models, but they are not yet large language models. The LLM label becomes appropriate only after the project introduces subword tokenization, a decoder-only Transformer, a meaningful parameter count, and training on a substantial corpus.

Keeping this distinction explicit prevents a small statistical model from being presented as something larger than it is.

The stable language-model loop

Across the project, the conceptual loop is:

```
text -> representation -> context
      -> next-symbol probabilities
      -> sampling -> generated text
```

Version 1 obtains probabilities directly from observed character counts. Later versions introduce trainable parameters, learning objectives, optimization, richer contexts, and new token representations.

The real Version 1 foundation

Version 1 is a character-level statistical language model written with standard Python. It reads an embedded English corpus, creates adjacent character pairs, counts transitions, normalizes counts into probabilities, and samples one character at a time.

Its implemented checks verify:

- the correct number of adjacent-character examples;
- next-character probabilities sum to 1;
- generation is reproducible with the same seed;
- the generated output has the requested length.

Version 1 uses only observed counts, probability normalization, and sampling. It has no trainable neural components.

Roadmap - Statistical and neural foundations

Version 1 - Character statistical model

Counts adjacent character transitions, converts counts into probabilities, and generates text through reproducible sampling with standard Python.

Version 2 - Small neural network with NumPy

Replaces the count table with a small neural model and makes parameter learning visible through NumPy operations.

Version 3 - Training foundations

Introduces batches, data splits, a training objective, optimization, and systematic reproducibility.

Version 4 - TensorFlow introduction

Rebuilds the small model with TensorFlow and automatic differentiation after the underlying calculations are understood.

Version 5 - Embeddings and context windows

Learns vector representations and uses more than one preceding character as context.

Roadmap - Sequence modeling and attention

Version 6 - Recurrent language model

Introduces a simple RNN and a hidden state that carries information through a sequence.

Version 7 - GRU or LSTM model

Improves sequence learning and training stability with gated recurrent units.

Version 8 - Attention

Builds scaled dot-product attention from understandable components and examines how tokens exchange information.

Version 9 - Transformer block

Combines causal self-attention, feed-forward layers, residual connections, and normalization.

Version 10 - Mini decoder-only language model

Stacks Transformer blocks and performs autoregressive generation with causal masking.

Roadmap - Data, training, and model use

Version 11 - Subword tokenizer and public datasets

Moves from characters to subword tokens and introduces documented open datasets.

Version 12 - Kaggle training pipeline

Adds efficient input pipelines, checkpoints, resumable runs, and mixed precision where the available hardware supports it.

Version 13 - Evaluation and controlled generation

Introduces evaluation appropriate to the trained model and generation controls such as temperature, top-k, and top-p.

Version 14 - Instruction tuning

Fine-tunes on small instruction datasets whose licenses support the intended workflow.

Version 15 - Advanced small LLM

Packages configuration, training, inference, documentation, model cards, and reproducible experiments.

Dataset strategy

Version 1 embeds a tiny educational corpus directly in the notebook. Later versions can first move to small text files and then to public corpora such as Tiny Shakespeare, WikiText, or comparable educational datasets.

Before training on a real dataset, the project records:

- source and exact dataset version;
 - license and allowed uses;
 - language and content scope;
 - cleaning and preprocessing decisions;
 - training, validation, and test splits.
-

Kaggle strategy

- notebooks execute from top to bottom;
 - seeds and configurations are explicit;
 - dataset paths are configurable under `/kaggle/input`;
 - advanced versions save checkpoints under `/kaggle/working`;
 - CPU and GPU configurations use conservative sizes;
 - a fast validation path remains available without a GPU.
-

Quality, safety, and documentation

Each version includes checks appropriate to its scope, explicit limitations, and clear error handling. Data splits and quantitative metrics are added when the model and training stage make them applicable.

Versions using real datasets include provenance and licensing notes. Trained-model versions include reproducible instructions and a model card describing intended use and limitations.

Expected final result

The expected result is an English decoder-only small LLM that learners can train or fine-tune on accessible Kaggle hardware, with a tokenizer, TensorFlow pipeline, checkpoints, controlled generation, appropriate evaluation, and documentation. The project remains centered on understanding and reproducibility rather than commercial-model scale.